

Customer App & Web Architecture Commerce and Delivery Experience

Detailed channel UX, authentication, discovery, cart, address validation, Paystack checkout, order tracking, notifications, privacy and monorepo implementation flow.

CUSTOMER APP

CUSTOMER WEB

MERCHANT

DRIVER

ADMIN

Architecture principles

- 1

One customer domain, two channels

App and web share contracts and business logic while keeping native and web UI separate.
- 2

Checkout revalidates everything

Store, items, prices, address, fees and payment intent are confirmed server-side.
- 3

Payment truth is provider-verified

The client can start checkout but cannot mark an order paid.
- 4

Orders carry immutable snapshots

Catalog and profile changes never rewrite historical transaction meaning.
- 5

Guest browsing stays useful

Authentication is required only for protected actions such as payment, tracking and account data.

DOCUMENT SCOPE

- 1

Role experience

Screens, commands, states and UX
- 2

Workflow ownership

Who reads, writes, approves and resolves
- 3

Data contracts

Canonical records, snapshots and enums
- 4

Security boundaries

Rules, Functions, App Check and audit
- 5

Monorepo flow

Shared packages without forced UI reuse
- 6

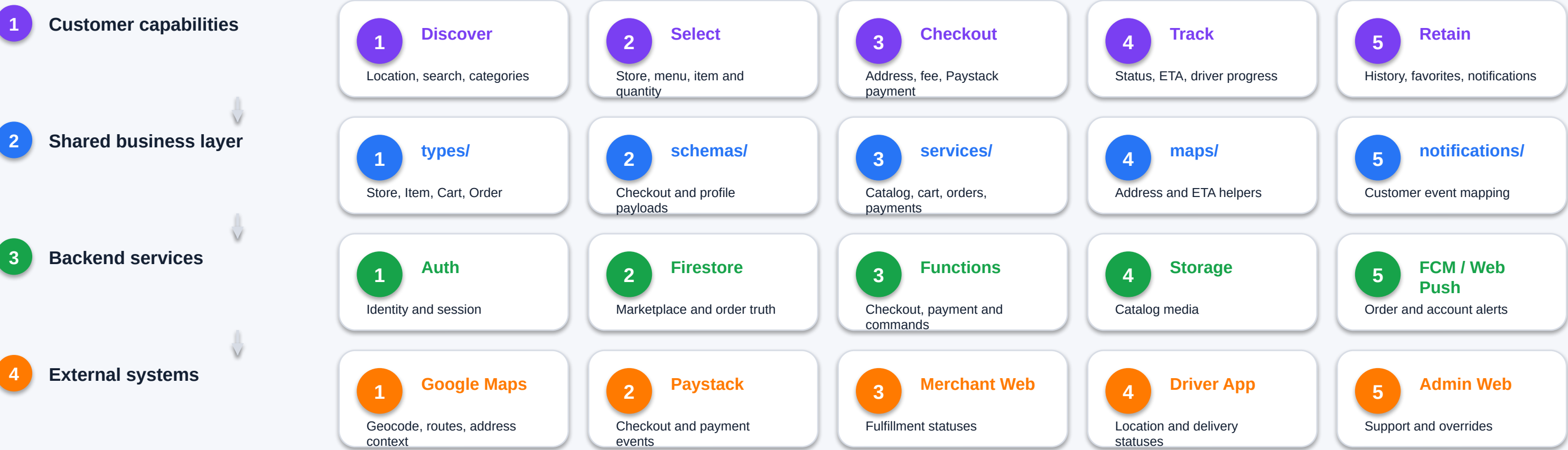
Development plan

Phases, tests, deployment and Codex prompts

VERSION 1.0 - JULY 2026

Customer Channels in the Five-App System

Customer App and Customer Web expose the same delivery marketplace through channel-appropriate interfaces.



Channel boundary

Share schemas, services and domain logic. Do not force React Native screens into Customer Web or web DOM components into the Expo application.

Information Architecture and Channel Adaptation

The same user journey is expressed through mobile-native navigation and responsive web layouts.

1 Customer App

2 Customer Web

1 Bottom navigation

Home, Orders, Cart and Account; notification entry from header.

2 Native interaction

Touch-first cards, sheets, gestures, safe areas and platform back behavior.

3 Long lists

Virtualized store, item, order and notification lists.

4 Payment return

Deep-link / app-resume reconciliation with canonical payment status.

1 Responsive navigation

Desktop header / sidebar where useful; bottom or compact nav on mobile web.

2 Browser semantics

Address bar routes, keyboard focus, shareable store/item URLs and accessible forms.

3 Checkout redirect / popup

Provider flow returns to a route that verifies and reloads canonical order state.

4 Progressive enhancement

Core browse/cart UX remains functional without heavy maps or animation.

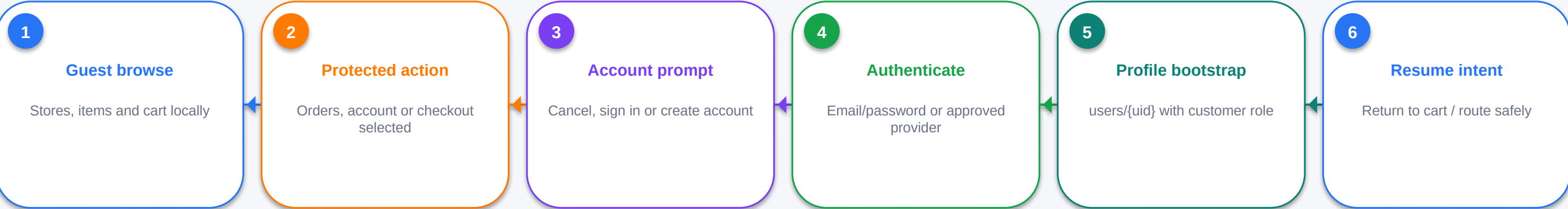
Shared visual language

Spaceman branding, money formats, status text, empty/error states and interaction terminology remain consistent across App and Web without requiring identical component implementations.

Guest Mode, Authentication and Account Lifecycle

Guest browsing reduces friction; protected operations require a verified user session.

1 Identity flow



2 Capability matrix

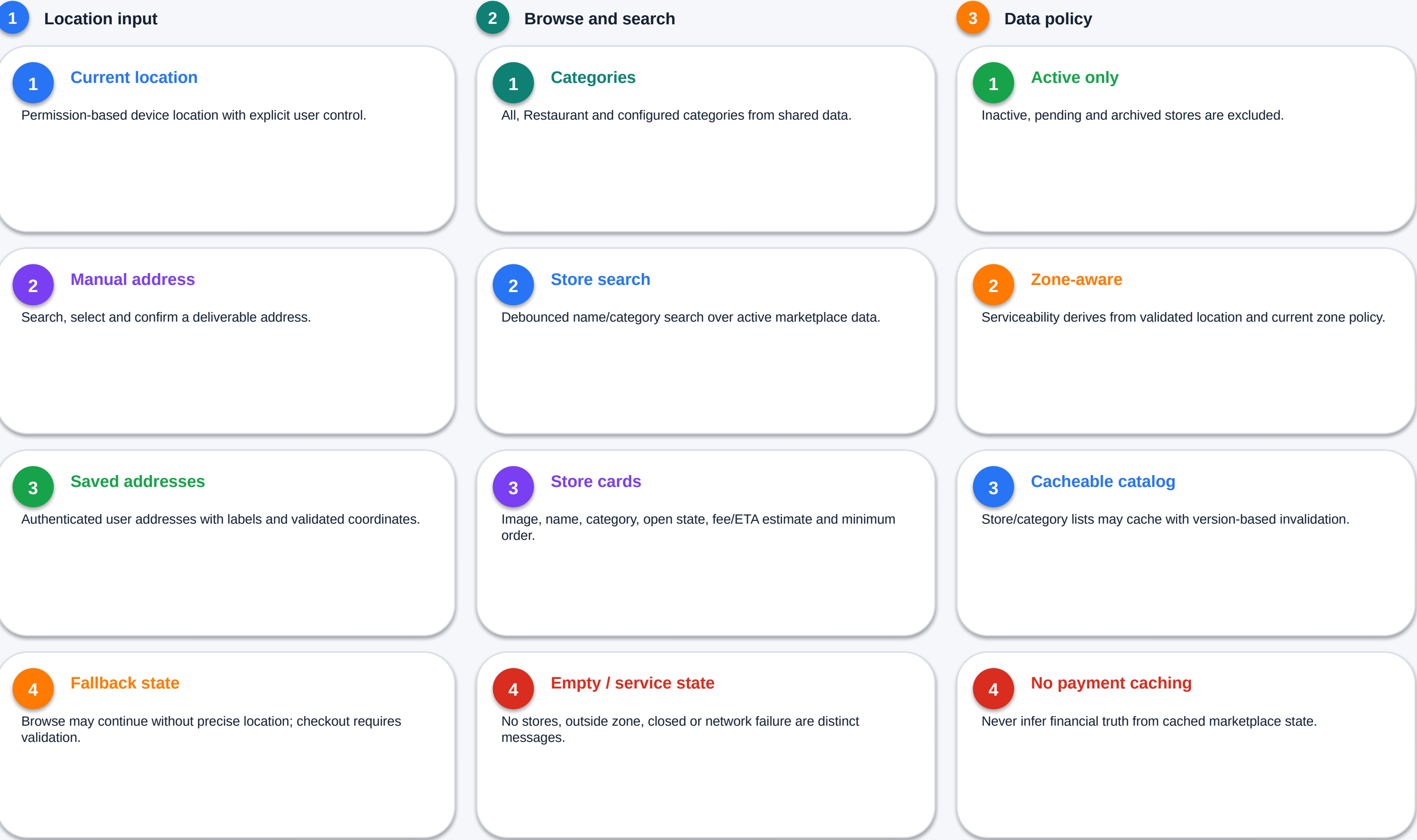
Capability	Guest	Authenticated customer	Backend rule
Browse stores/items	Yes	Yes	Public active catalog only
Use local cart	Yes	Yes	Client state; revalidate at checkout
Favorites	Optional local	Synced to user	Own user scope
Start payment	No	Yes	Valid Auth + App Check + checkout command
Track / view history	No	Own orders	customerId == uid
Edit profile	No	Allowed fields only	Role/status immutable
Delete account request	No	Submit request	Archive/redact workflow

Current deletion decision

Approved deletion disables sign-in, archives the user, redacts personal fields and clears disposable data while retaining required order, payment and audit records with non-identifying references.

Location, Search and Marketplace Discovery

Discovery presents only stores that are active, serviceable and currently relevant to the customer location.



Store, Menu, Item Details and Favorites

Catalog browsing is fast and visual, while order-critical values are revalidated at checkout.

1 Store and menu experience

1 Store header

Hero/card imagery, name, category, description, open state and delivery context.

2 Menu tab

Available items grouped/searchable by category, with image, description and price.

3 Info tab

Address, operating hours, minimum order and service information.

4 Item details

Large image, description, modifiers when supported, quantity and add-to-cart.

2 Favorites and integrity rules

1 Favorite item

Own-user reference to itemId/storeId; remove gracefully if item is retired.

2 Display snapshot

Catalog is mutable; cart stores a provisional display snapshot only.

3 Availability

Unavailable items cannot be added; existing cart lines require checkout revalidation.

4 Price changes

Show clear update before payment; never silently charge a different total.

5 Image behavior

Responsive sizes, placeholders, lazy loading and meaningful alt text.

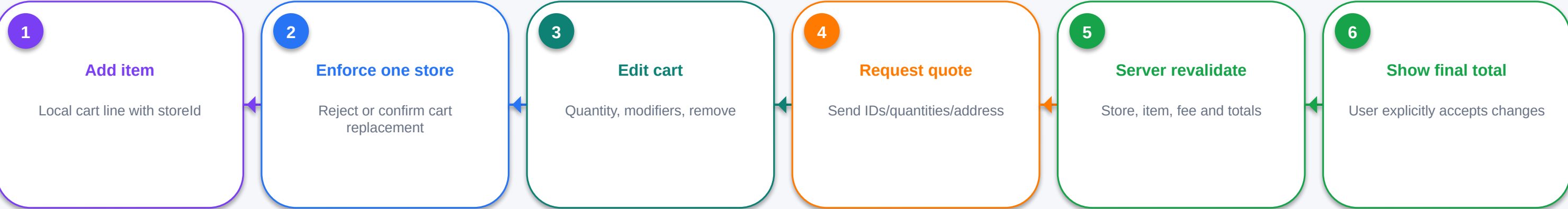
Historical rule

When an order is created, item ID, name, quantity, unit price, modifiers and presentation values are snapshotted into orders/{orderId}; later item edits do not change them.

Cart State, Pricing and Pre-Checkout Revalidation

The cart is a convenience model; the backend produces the authoritative checkout quote.

1 Cart to quote flow



2 Cart and quote contract

Element	Client role	Server role	Change handling
Store	Display and one-store guard	Validate active/open/serviceable	Block checkout if unavailable
Items	Provisional lines	Load current price/availability	Return removed or changed lines
Subtotal	Estimate	Recalculate from authoritative items	Display difference
Delivery fee	Estimate after location	Apply versioned fee rule + distance	Snapshot rule/version
Discount / service fee	Display	Validate policy and eligibility	Reject invalid promotion
Total	Display only	Authoritative money calculation	Customer reconfirms
Quote expiry	Countdown hint	Server expiry timestamp	Refresh before payment

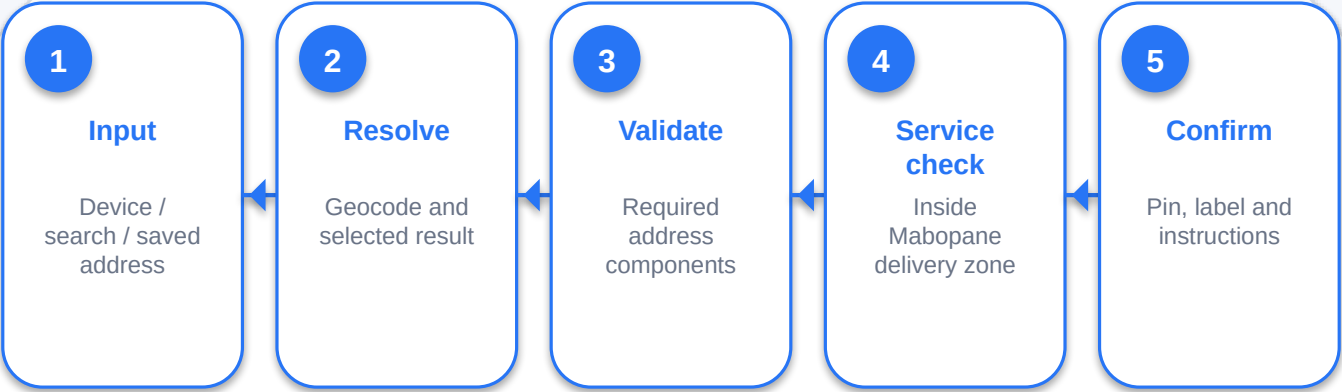
Money representation

Store monetary values as integer minor units or a single approved decimal strategy across all apps and Functions. Formatting is a presentation concern; calculations are shared and tested.

Address Validation, Serviceability, Distance and ETA

Google services enrich a validated delivery address through one normalized backend path.

1 Address workflow



- Store address snapshot and customer delivery address snapshot remain separate.
- Capture latitude/longitude, formatted address, place reference/source and validation timestamp.
- Let the user correct a map pin or add delivery instructions without corrupting the validated address.
- Never expose provider API keys with unrestricted server privileges in a client bundle.

2 Route and fee enrichment

- 1

Route / route matrix
Backend obtains distance and ETA between selected store and delivery destination.
- 2

Normalized result
distanceKm, routeEtaMinutes, source, calculatedAt and optional route reference.
- 3

Fee rule
 $\text{Base} + \text{excess distance} \times \text{rate} + \text{conditional surcharge}$, clamped to min/max.
- 4

Freshness
Quote-time ETA is an estimate; tracking ETA may refresh under a controlled policy.
- 5

Failure state
Do not fabricate a route or fee. Block payment or use an explicitly approved fallback.

V1 launch constraint

One Mabopane delivery area, one standard delivery type and no surge pricing. Expand only after the base workflow and operational data are stable.

Paystack Checkout, Verification and Webhook Flow

The client initiates payment; only trusted backend verification can create paid fulfillment value.

1 Payment sequence



2 Payment state model

State	Customer UI	Authoritative transition	Fulfillment
draft_quote	Review final total	Checkout service	No
payment_processing	Provider UI / waiting	Initialize + callback reference	No
paid	Success / order created or activated	Verified transaction / charge.success	Yes
payment_failed	Retry / change method	Provider verification/webhook	No
payment_abandoned	Resume or retry	Expiry/reconciliation	No
refund_pending	Status and support context	Admin refund command	No new fulfillment
refunded	Refund complete	Provider webhook / verify	Closed

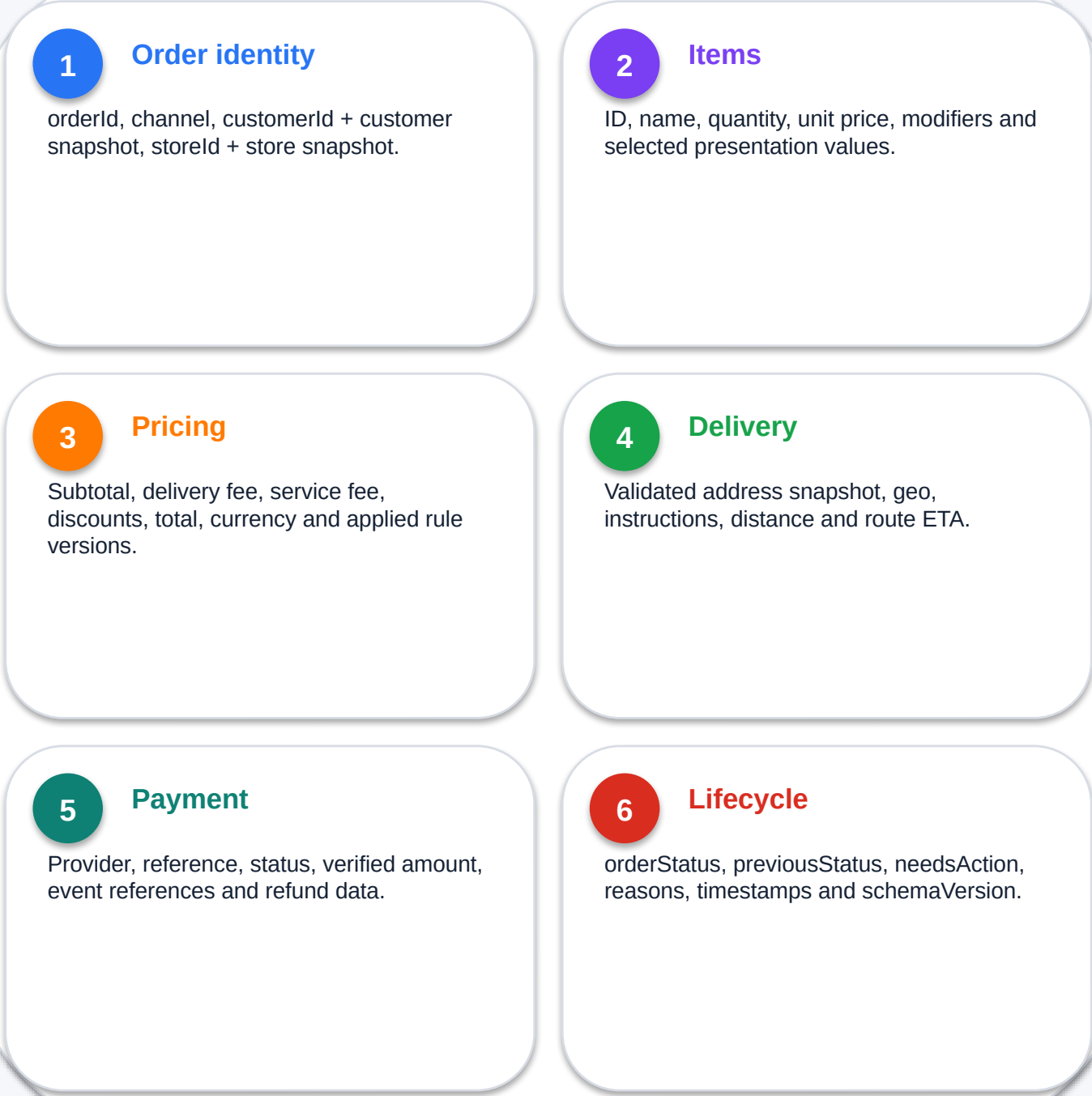
Provider rule

Paystack advises initializing transactions from the backend, keeping the secret key off frontends, verifying status and amount, and not treating a visited callback URL as proof of success.

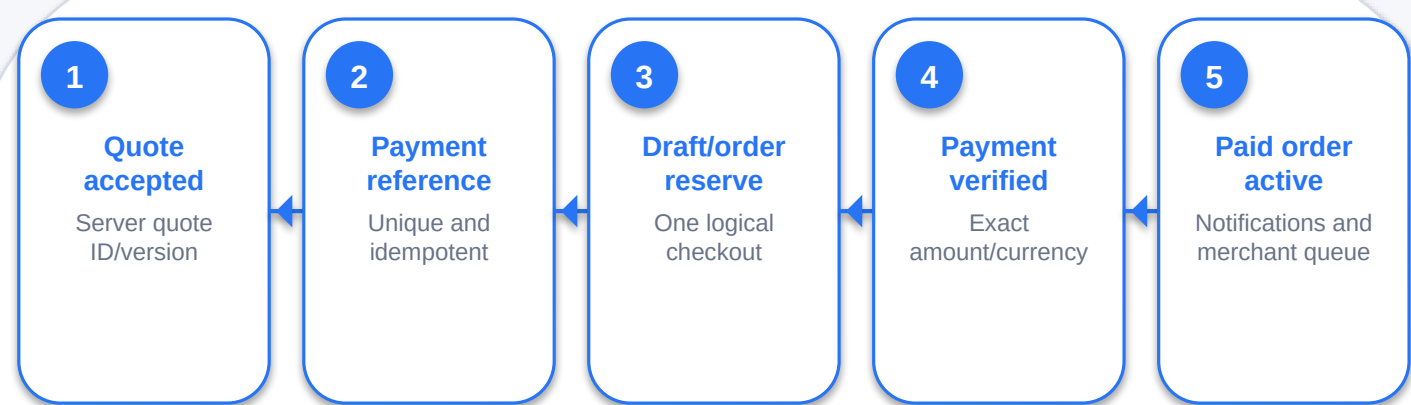
Canonical Order Creation and Immutable Snapshots

An order is a historical transaction record plus a live operational state machine.

1 Order snapshot sections



2 Creation consistency



- Use a stable checkoutId/orderId/reference mapping so retries cannot create duplicate paid orders.
- Write order status, payment event, audit/status history and notification outbox atomically where feasible.
- A client interruption after payment must recover by reference reconciliation, not create a second transaction.
- Server timestamps are authoritative for every SLA and lifecycle calculation.

Decision to confirm in repo

The exact moment when orders/{orderId} is first created may be pre-payment or post-verification. Either model is valid only if idempotency, abandoned-payment cleanup and canonical references are explicit.

Order Status, ETA and Driver Tracking

Tracking combines canonical status history with controlled location and route updates.

1 Visible lifecycle



2 Customer tracking contract

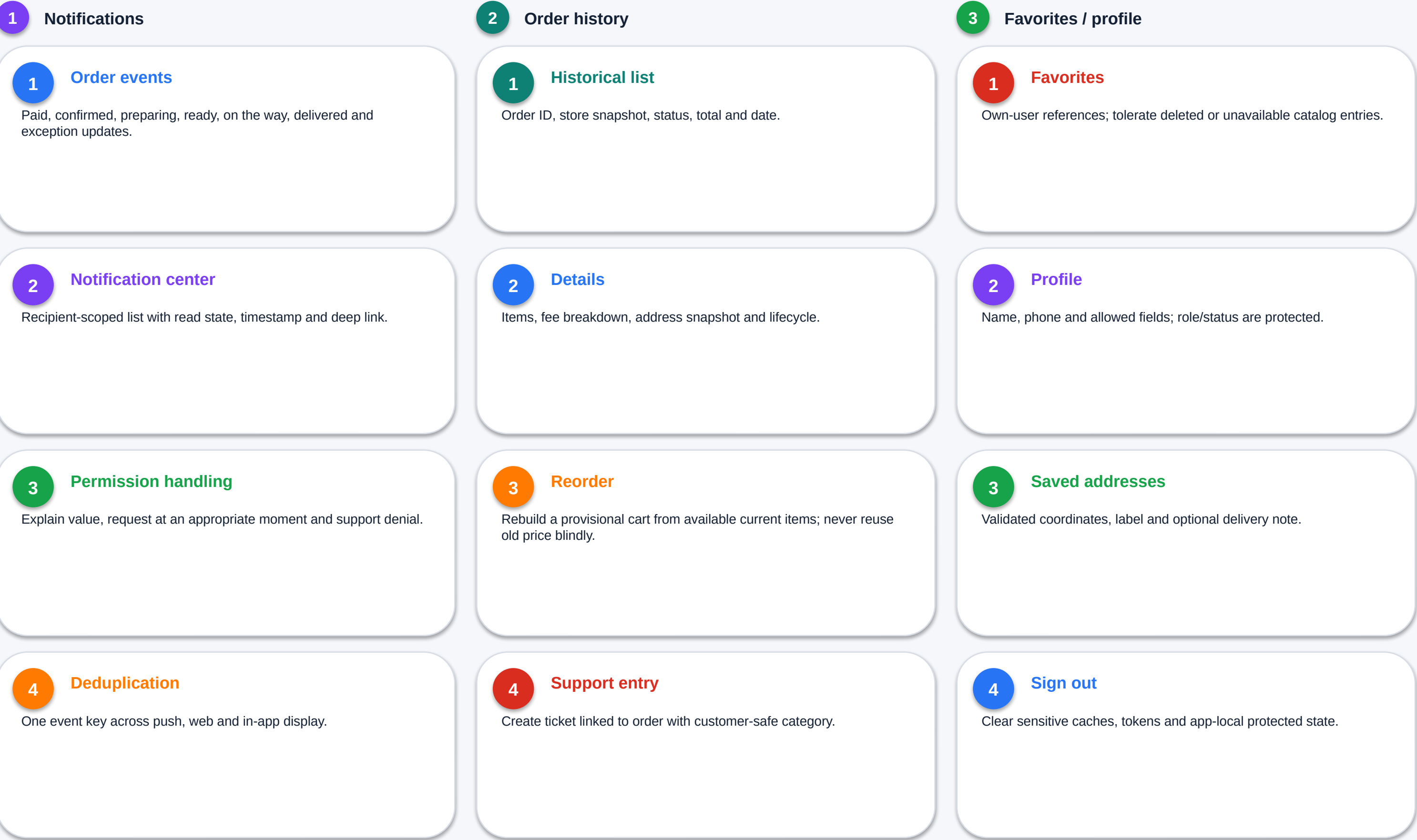
UI element	Source	Update strategy	Privacy / fallback
Progress timeline	orderStatus + statusHistory	Realtime active order listener	Text remains usable without map
ETA	delivery.routeEtaMinutes / refreshed route	Controlled refresh, not every render	Show estimate and timestamp
Driver location	Scoped live location projection	Throttled while on the way	Stop access after completion
Store / driver identity	Order snapshots and assignment projection	Update on assignment	Do not expose private contact details
Issue banner	needsAction + customer-safe reason	Realtime	Avoid internal-only diagnostics
Proof / delivered time	deliveredAt + proof policy	Final canonical update	Retained in history

Map degradation

A stale or unavailable map must not erase the status timeline. Show last update time and a clear fallback rather than presenting old coordinates as live.

Notifications, History, Favorites and Reordering

Retention features are projections over the canonical catalog and order history, not parallel sources of truth.



Cancellations, Support and Privacy Requests

Customer requests are recorded as structured activities and resolved through policy-aware server commands.

1 Customer requests

1 Cancel request

Allowed only in policy-approved states; backend determines financial effect.

2 Report order issue

Missing item, wrong item, late order, address/contact problem or other category.

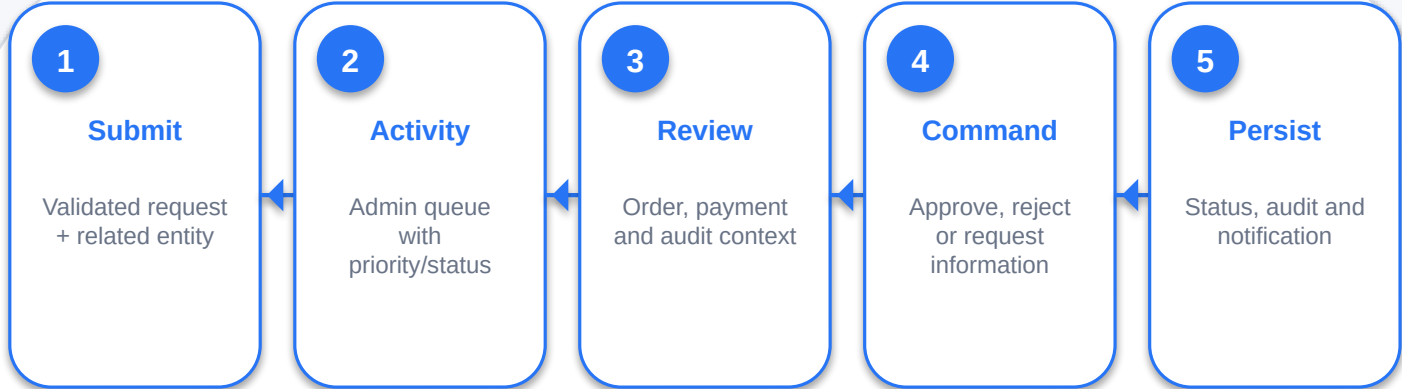
3 Refund visibility

Show refund pending / refunded provider-confirmed state without promising timing.

4 Delete account request

Reason optional; creates a reviewable activity and user notification.

2 Resolution workflow



- Do not let the client directly cancel a paid order outside the allowed transition policy.
- Support content should not expose internal fraud, payment or driver security diagnostics.
- Account deletion clears cart, favorites and tokens while preserving legally/operationally required records.
- Suspension is reversible and separate from archive/redact deletion.

Financial boundary

A customer-facing refund request is not a refund result. The result becomes final only after the backend and Paystack confirm it.

Customer Data Access, App Check and Abuse Controls

Customer authorization is ownership-based and enforced independently of UI navigation.

1 Authorization

1 Own profile

Read/update only allowed self fields.

2 Own orders

customerId must equal authenticated uid.

3 Own notifications

recipientUid must equal uid.

4 Public catalog

Only active customer-visible store/item fields.

2 Protection layers

1 Route guards

Protected screens require session; guest prompt preserves intent.

2 Rules

Deny direct protected field writes and cross-user access.

3 Functions

Checkout, payment, cancellation and deletion commands.

4 App Check

Reduce abuse from unauthorized app clients; still require Auth and rules.

3 Privacy controls

1 Location minimization

Store only data required for delivery/history policy.

2 Token cleanup

Remove notification tokens on sign-out/device invalidation/deletion.

3 Contact protection

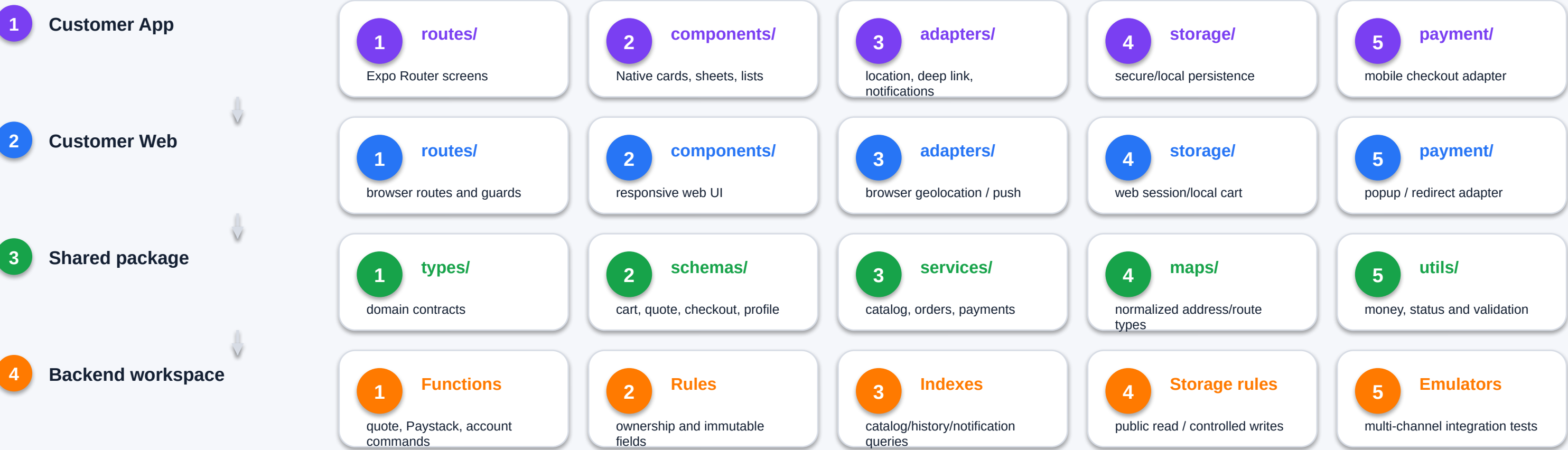
Avoid exposing raw driver/merchant personal contact details.

4 Audit

Sensitive profile/deletion/support actions are traceable.

Customer App and Web Monorepo Boundaries

The two customer channels share domain packages but keep platform-specific navigation, UI, storage and payment adapters.



Dependency direction

apps/customer-app and apps/customer-web may import packages/shared. Shared cannot import either app, and the two apps cannot import each other.

Performance, Offline Behavior and Accessibility

Mobile and web optimize for low-bandwidth use without weakening transaction correctness.

1 Performance

1 Virtualized lists

Stores, items, orders and notifications render bounded visible rows.

2 Images

Responsive/compressed assets, thumbnails and lazy loading.

3 Listeners

Realtime only for active tracking and essential notifications.

4 Maps

Lazy-load and stop location rendering when not needed.

2 Reliability

1 Offline catalog

Cached browsing may continue with stale-data indication.

2 Checkout fail closed

Payment and quote require network and fresh server validation.

3 Resume reconciliation

App/web return resolves payment/order by stable reference.

4 Crash-safe cart

Persist provisional cart with schema version and invalidation.

3 Accessibility

1 Screen readers

Labels, semantic headings and status descriptions.

2 Touch / keyboard

44 px mobile targets and complete keyboard web flow.

3 Contrast / motion

Status not color-only; respect reduced motion.

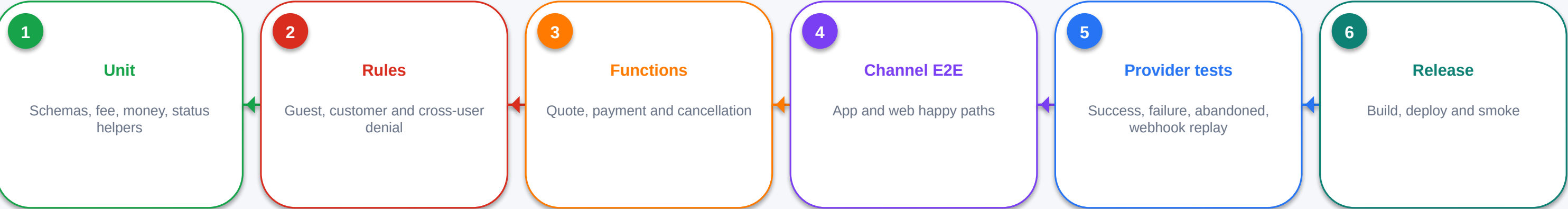
4 Errors

Plain-language recovery and no lost form intent.

Customer Test Matrix and Production Gates

Critical money and ownership paths require emulator, provider-test and end-to-end evidence.

1 Verification sequence



2 Acceptance scenarios

Area	Required scenario	Pass condition
Guest	Browse, cart, protected route prompt	Intent preserved; no protected reads
Checkout	Price/item/store/zone changed before pay	User sees change or checkout blocks
Paystack success	Callback arrives before webhook or vice versa	Exactly one paid order
Paystack failure	Failed/abandoned payment	No merchant fulfillment
Tracking	Merchant and driver update order	Customer sees ordered timeline and safe map
Privacy	Attempt another user's order/notification	Denied by rules
Account deletion	Approved archive/redact	Login disabled; disposable data gone; history retained

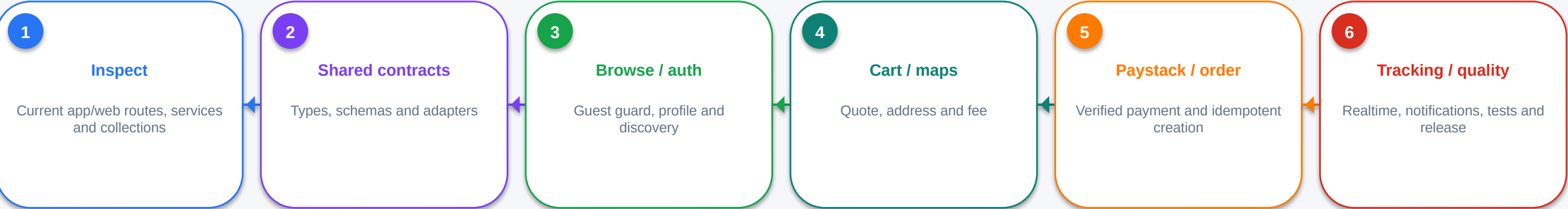
Release gate

Typecheck, lint, web build, Expo checks/build, emulator rules/functions, test payment reconciliation and manual mobile/browser smoke must pass for the affected scope.

Customer Development Roadmap

Build from stable catalog browsing toward verified payment and live tracking without broad rewrites.

1 Phased implementation



2 Phase exit criteria

Phase	Exit criteria	Do not proceed if
1. Discovery	Actual repo/deployed truth documented	Collection/function assumptions unresolved
2. Contracts	Both channels compile against shared contracts	UI coupling introduced
3. Browse/auth	Guest and authenticated flows tested	Protected data readable by guest
4. Quote/maps	Authoritative quote and serviceability stable	Client invents fee or ETA
5. Payment/order	Exactly one paid order per provider reference	Callback can mark paid
6. Tracking/release	Cross-app states, privacy and builds pass	Stale listeners or unverified payment states remain

Reusable Codex instruction

Resume current progress. Inspect customer-app, customer-web and shared backend first. Implement one bounded feature, preserve guest browsing and one Firebase project, validate payment server-side, run affected tests/builds and report all files/schema/deployment changes.

Customer App & Web Traceability and Decisions

Traceability, current decisions and implementation assumptions.

1

Source-to-module traceability

Source	Captured architecture detail	Used in this document
Admin blueprint - system context	Five apps, shared business layer and one backend	Pages 2, 15
Current deployed customer experience	Home, store/menu/info, guest guard, cart, orders, notifications, account and favorites	Pages 3-6, 11-13
Admin blueprint - fee setup	Mabopane rule, Maps distance and checkout visibility	Pages 7-9
Admin blueprint - order lifecycle	Customer order initiation and cross-channel tracking	Pages 9-11
Admin blueprint - security	Ownership, Functions, App Check and archive/redact	Pages 4, 13-17
Official Paystack guidance	Backend initialization, verification, amount checks and webhooks	Page 9
Google Maps platform	Geocoding/route services normalized through backend adapters	Page 8

2

Normalized decisions

Two UI stacks

Expo app and React web share domain logic, not forced components.

Guest mode

Browse/cart remain open; checkout, history and account require authentication.

Payment truth

Verified Paystack state is required before fulfillment.

One zone v1

Mabopane standard delivery with no surge.

Order snapshots

Historical customer/store/item/pricing data remain immutable.

Deletion

Archive/redact and Auth disable preserve required history.

Source precedence

Current repo/deployment > approved architecture > mockup intent.

Document status

Architecture synthesis for implementation. Before changing code, validate exact collection names, existing Cloud Functions, deployed rules, current routes and environment variables against the monorepo.